

LogiSphere: Cloud-Native Logistics and Supply Chain Management Platform

CNAD Project Review-1



Under the Guidance of

Dr. P. Srilatha

Associate Professor

Department of CSE,

KLH CSE Bowrampet Campus



Team Members

B. Hemalatha – 2420030698

Bhagyalaxmi – 2420030700

D. Yashaswini – 2420030016

Outline of Project Work



Problem Statement

- **Complex coordination:** logistics operations involve suppliers, warehouses, inventory, orders, shipments, and delivery – all needing tight coordination.
- **Limited coordination and delayed information:** traditional systems often struggle to share real-time status across these interconnected activities.
- **Inventory inconsistencies:** disconnected record-keeping across warehouses can lead to stock mismatches and fulfilment errors.
- **Difficulty scaling individual operations:** monolithic applications make it hard to scale one function (e.g. order processing) independently of the others.
- **Need identified:** a flexible, scalable, cloud-native platform that can manage and coordinate logistics activities efficiently.

Objectives

- Design and develop LogiSphere, a digital platform to manage and track products, inventory, orders, shipments, and deliveries across the supply chain.
- Enable suppliers to register products, warehouses to manage inventory, customers to place and track orders, and admins to monitor operations from a centralized dashboard.
- **Build the system as independent microservices** (Order, Inventory, Shipment, Delivery, User/Auth) using Spring Boot, so each service can be developed and scaled on its own.
- **Use Spring Cloud Gateway and Eureka** for request routing and dynamic service discovery across the platform.
- **Enable service communication** through REST APIs and Apache Kafka, allowing business functions to operate independently and efficiently.
- Use MySQL for data storage and React.js for a responsive, web-based user interface.
- **Containerize services with Docker and orchestrate them with Kubernetes** for independent deployment, scaling, and recovery, supported by Docker Compose for local development.

Literature Survey

Authors	Title of the Paper	Methodology	Inferences	Limitation / Future Work
Sharma & Patel [1]	Microservices Architecture for Optimizing Warehouse Management Systems in Global Supply Chains, ResearchGate, 2025	Decomposes traditional WMS functions into domain-specific microservices (Core, Inbound, Outbound); evaluated via multi-regional warehouse case studies.	Microservices improve scalability, resilience and agility of warehouse operations, with measurable reductions in operational overhead across geographies.	Focused mainly on warehouse operations; does not deeply cover event-driven, adaptive decision-making across the wider order-to-delivery chain.
Terzidis et al. [2]	A Microservice-based Framework for Integrating IoT, Semantic and AI Services for Supply Chain Management, ScienceDirect, 2019	Integrated microservice architecture consuming IoT data feeds, semantically annotating them for supply chain monitoring, location analysis and delay estimation.	Microservices combined with IoT and semantic annotation enable real-time supply-chain monitoring and better visibility of goods in transit.	Emphasis on semantic/IoT integration rather than containerized deployment, service discovery or API-gateway based scalability.
Čilić, Krivić, Podnar Žarko & Kušek [3]	Performance Evaluation of Container Orchestration Tools in Edge Computing Environments, Sensors (MDPI), 2023	Comparative evaluation of Kubernetes, K3s, KubeEdge and ioFog orchestration platforms against QoS parameters in real network/execution environments.	Kubernetes and its lightweight distributions provide effective, adaptable scheduling and are well suited for scalable service orchestration.	Focused on edge-computing scenarios; does not evaluate orchestration specifically for logistics/supply-chain microservice workloads.
Open-Source Toolchain Review [4]	Microservices and Real-Time Processing in Retail IT: A Review of Open-Source Toolchains and Deployment Strategies, arXiv, 2025	Reviews open-source tools (Kafka, Spring Boot) for building event-driven, real-time microservice pipelines in retail IT deployments.	Kafka combined with Spring Boot enables asynchronous, decoupled communication between services, supporting scalable and responsive real-time retail systems.	Reviews toolchains at a conceptual level; lacks a working end-to-end deployment on Kubernetes with monitoring and CI/CD.

Problem Gap Identification

- **Amazon / Flipkart:** large-scale commercial platforms with sophisticated but closed, proprietary logistics engines tightly bound to their own marketplace and payments systems.
- **Typical academic projects:** usually stop at basic CRUD operations – Add Product, Delete Product, Create Order – without demonstrating real service adaptiveness.
- **Identified gap:** there is no widely available, reusable, adaptive cloud-native logistics architecture built purely around independent, well-defined services.
- **Missing capability:** existing systems mostly record transactions; they do not automatically respond to operational conditions such as insufficient stock, delivery overload, or shipment delay.
- **How LogiSphere addresses this:** logistics functions are exposed as independent microservices with an event-driven Adaptive Decision layer that reacts to real-time conditions.

Innovation, Creativity, and Novelty

- **Adaptive warehouse selection:** instead of returning “Out of stock,” the system automatically checks other registered warehouses and recommends/fulfils the order from one that has sufficient stock.
- **Event-driven architecture:** Kafka-based communication (OrderCreated, StockLow, ShipmentDelayed events) keeps services loosely coupled and independently scalable.
- **Smart delivery assignment:** delivery personnel are recommended based on real-time distance and current active-delivery workload rather than a fixed, static rule.
- **Proactive delay handling:** shipment delays are detected automatically and trigger instant customer notifications, without manual tracking.
- **Operational-intelligence dashboard:** the LogiSphere Control Center surfaces live alerts and recommendations (low stock, delayed shipments, suggested fulfilment) instead of just static counts.

Feasibility and Project Plan

- **Phase 1 – Foundation:** project setup, GitHub repository, Order Service, MySQL, REST APIs, Postman testing.
- **Phase 2 – Core Microservices:** Inventory, Shipment, Delivery, User/Auth, and Notification services.
- **Phase 3 – Service Integration:** Eureka service discovery, Spring Cloud API Gateway, inter-service REST and Kafka communication.
- **Phase 4 – Adaptive Features:** low-stock detection, warehouse selection, delivery-person assignment, shipment-delay detection, admin alerts.
- **Phase 5 – Security:** Spring Security with JWT authentication and role-based authorization.
- **Phase 6 – Frontend:** React dashboards for Customer, Admin, Warehouse, and Delivery views.
- **Phase 7 – Cloud-Native Deployment:** Dockerize all services, Docker Compose, and Kubernetes orchestration.
- **Phase 8 – Monitoring:** Prometheus and Grafana for service health and performance monitoring.

Technologies Used

- **Backend & Services:** Java, Spring Boot – independent microservices for Order, Inventory, Shipment, Delivery and User/Auth.
- **Frontend:** React.js – customer, admin, warehouse and delivery dashboards.
- **Service Communication:** REST APIs for direct calls; Apache Kafka for asynchronous, event-driven communication.
- **Data Layer:** MySQL for persistent storage (service-specific databases); Redis for caching frequently accessed data.
- **Containerization & Orchestration:** Docker and Docker Compose for local development; Kubernetes for deployment, scaling and recovery.
- **DevOps & CI/CD:** Git & GitHub for version control; GitHub Actions for automated build/test/deploy pipelines.
- **Monitoring & Quality:** Prometheus and Grafana for performance/health monitoring; SonarQube for code quality; Trivy for container vulnerability scanning.

Conclusion

- LogiSphere provides a scalable and reliable cloud-native solution for managing logistics and supply chain operations end-to-end.
- **Microservices architecture** (Order, Inventory, Shipment, Delivery, User/Auth) enables independent development, deployment, and scaling of each business function.
- **Containerization and orchestration** with Docker and Kubernetes allow services to be deployed, managed, and scaled reliably across environments.
- **APIs and event-driven communication** via REST and Apache Kafka keep services loosely coupled while enabling real-time coordination.
- **Monitoring and quality tooling** (Prometheus, Grafana, SonarQube, Trivy) support performance visibility, code quality, and secure deployments.
- Overall, the project demonstrates the practical application of modern cloud-native application development and container orchestration principles to a real-world logistics use case.

Thank You